

Python Programming

Day 12: Stock Portfolio Analyzer

OOP, CSV, Pandas, Matplotlib, File Output

Contents

1 프로젝트 개요	2
2 파일 구조	2
3 Input Data	2
3.1 data/portfolio.csv	2
4 Stock Class	3
4.1 stock.py	3
5 Portfolio Class	3
5.1 portfolio.py	3
6 Portfolio Analysis	4
6.1 analysis.py	4
7 Visualization	4
7.1 visualization.py	4
8 Main Program	5
8.1 main.py	5
9 Output Files	5
9.1 output/portfolio_analysis.csv	6
10 Day 12 핵심 요약	6

1. 프로젝트 개요

프로젝트 목표

Day 12에서는 주식 포트폴리오 데이터를 CSV 파일에서 읽고, 객체 지향 구조로 종목과 포트폴리오를 표현한 뒤, Pandas를 이용해 분석하고 Matplotlib으로 시각화한다. 최종 분석 결과는 CSV와 PNG 파일로 저장한다.

핵심 포인트

프로그램의 전체 흐름은 CSV 입력 → Stock 객체 생성 → Portfolio 계산 → Pandas 분석 → Matplotlib 시각화 → 결과 저장이다.

2. 파일 구조

프로젝트 구성

```
Day12/
|
|-- main.py
|-- stock.py
|-- portfolio.py
|-- analysis.py
|-- visualization.py
|
|-- data/
|   |-- portfolio.csv
|
|-- output/
|   |-- portfolio_analysis.csv
|   |-- return_rates.png
```

- stock.py: 주식 한 종목의 데이터와 계산을 담당한다.
- portfolio.py: 여러 Stock 객체를 관리하고 전체 포트폴리오 값을 계산한다.
- analysis.py: Portfolio 데이터를 DataFrame으로 변환하고 수익률을 분석한다.
- visualization.py: 종목별 수익률을 막대그래프로 시각화하고 이미지로 저장한다.
- main.py: 입력, 객체 생성, 분석, 저장, 시각화의 전체 흐름을 연결한다.
- data/portfolio.csv: 포트폴리오의 입력 데이터를 저장한다.
- output/: 분석 결과 CSV와 그래프 이미지를 저장한다.

3. Input Data

3.1. data/portfolio.csv

```
1 ticker,shares,buy_price,current_price
2 AAPL,10,180,210
3 MSFT,5,400,450
```

```
4 NVDA,8,120,150
5 GOOGL,6,170,160
6 TSLA,4,300,250
```

4. Stock Class

4.1. stock.py

```
1 class Stock:
2     def __init__(self, ticker, shares, buy_price, current_price):
3         self.ticker = ticker
4         self.shares = shares
5         self.buy_price = buy_price
6         self.current_price = current_price
7
8     def get_investment(self):
9         return self.shares * self.buy_price
10
11    def get_current_value(self):
12        return self.shares * self.current_price
13
14    def get_profit(self):
15        return self.get_current_value() - self.get_investment()
16
17    def get_return_rate(self):
18        return (self.get_profit() / self.get_investment()) * 100
```

5. Portfolio Class

5.1. portfolio.py

```
1 class Portfolio:
2     def __init__(self):
3         self.stocks = []
4
5     def add_stock(self, stock):
6         self.stocks.append(stock)
7
8     def get_total_investment(self):
9         total = 0
10        for stock in self.stocks:
11            total += stock.get_investment()
12        return total
13
14    def get_total_current_value(self):
15        total = 0
16        for stock in self.stocks:
17            total += stock.get_current_value()
18        return total
19
20    def get_total_profit(self):
21        return self.get_total_current_value() - self.get_total_investment()
22
23    def get_total_return_rate(self):
24        return (self.get_total_profit() / self.get_total_investment()) * 100
```

6. Portfolio Analysis

6.1. analysis.py

```
1 import pandas as pd
2
3
4 def create_dataframe(portfolio):
5     data = []
6
7     for stock in portfolio.stocks:
8         data.append({
9             "ticker": stock.ticker,
10            "shares": stock.shares,
11            "investment": stock.get_investment(),
12            "current_value": stock.get_current_value(),
13            "profit": stock.get_profit(),
14            "return_rate": stock.get_return_rate()
15        })
16
17     df = pd.DataFrame(data)
18     return df
19
20
21 def analyze_portfolio(df):
22     sorted_df = df.sort_values("return_rate", ascending=False)
23
24     best_stock = sorted_df.iloc[0]
25     worst_stock = sorted_df.iloc[-1]
26
27     print("=== Portfolio Analysis ===")
28     print(f"Best Stock: {best_stock['ticker']} ({best_stock['return_rate']:.2f}%)")
29     print(f"Worst Stock: {worst_stock['ticker']} ({worst_stock['return_rate']:.2f}%)")
```

7. Visualization

7.1. visualization.py

```
1 import matplotlib.pyplot as plt
2
3
4 def plot_returns(df):
5     x = df["ticker"]
6     y = df["return_rate"]
7
8     plt.bar(x, y)
9     plt.xlabel("Ticker")
10    plt.ylabel("Return Rate (%)")
11    plt.title("Return Rate (%) by Ticker")
12    plt.axhline(0)
13    plt.savefig("output/return_rates.png")
14    plt.show()
```

8. Main Program

8.1. main.py

```

1 import pandas as pd
2
3 from stock import Stock
4 from portfolio import Portfolio
5 from analysis import create_dataframe, analyze_portfolio
6 from visualization import plot_returns
7
8
9 df = pd.read_csv("data/portfolio.csv")
10
11 portfolio = Portfolio()
12
13 for _, row in df.iterrows():
14     stock = Stock(
15         row["ticker"],
16         row["shares"],
17         row["buy_price"],
18         row["current_price"]
19     )
20
21     portfolio.add_stock(stock)
22
23
24 print("=== Portfolio Summary ===")
25 print(f"Total Investment: {portfolio.get_total_investment()}")
26 print(f"Total Current Value: {portfolio.get_total_current_value()}")
27 print(f"Total Profit: {portfolio.get_total_profit()}")
28 print(f"Total Return Rate: {portfolio.get_total_return_rate():.2f}%")
29
30 print()
31
32 analysis_df = create_dataframe(portfolio)
33
34 print(analysis_df)
35 print()
36
37 analyze_portfolio(analysis_df)
38
39 analysis_df.to_csv(
40     "output/portfolio_analysis.csv",
41     index=False
42 )
43
44 plot_returns(analysis_df)

```

9. Output Files

프로그램을 실행하면 output/ 폴더에 분석 결과와 그래프가 저장된다.

- portfolio_analysis.csv: 종목별 투자금액, 현재 평가금액, 손익, 수익률을 저장한다.
- return_rates.png: 종목별 수익률 막대그래프를 저장한다.

9.1. output/portfolio_analysis.csv

```
1 ticker,shares,investment,current_value,profit,return_rate
2 AAPL,10,1800,2100,300,16.666666666666664
3 MSFT,5,2000,2250,250,12.5
4 NVDA,8,960,1200,240,25.0
5 GOOGL,6,1020,960,-60,-5.88235294117647
6 TSLA,4,1200,1000,-200,-16.666666666666664
```

10. Day 12 핵심 요약

- CSV 파일에서 포트폴리오 입력 데이터를 읽어 DataFrame으로 변환한다.
- CSV의 각 행을 Stock 객체로 변환하여 실제 데이터와 객체 지향 구조를 연결한다.
- Stock 클래스는 한 종목의 투자금액, 현재 평가금액, 손익, 수익률 계산을 담당한다.
- Portfolio 클래스는 여러 Stock 객체를 리스트로 관리하고 전체 포트폴리오 값을 계산한다.
- 함수에 전달된 객체는 필요한 속성과 메서드를 가지고 있으면 해당 기능을 사용할 수 있다.
- Pandas DataFrame으로 객체의 계산 결과를 표 형태로 구성한다.
- `sort_values()`와 `iloc`를 이용해 최고 및 최저 수익률 종목을 찾는다.
- Matplotlib의 `bar()`를 이용해 종목별 수익률을 시각화한다.
- `to_csv()`와 `savefig()`를 이용해 분석 결과와 그래프를 파일로 저장한다.
- Day 12에서는 OOP, CSV, Pandas, Matplotlib을 하나의 미니프로젝트 안에서 통합한다.